

Computational Complexity, Homework Problem 2, Fall 2024

Name: Heorhii Lopatin

ID: 456366

Date: November 21, 2024

Problem 1: Consider a language consisting of pairs of trees (T_1, T_2) such that T_1 is of radius at most 2 and T_2 contains a subgraph isomorphic to T_1 . Prove that this language belongs to the class **L**.

*Note: You do not need to argue that checking input syntax (i.e., checking if the input indeed consists of two graphs T_1 and T_2) can be done in **L**. However, observe that you need to reject inputs where T_1 and T_2 are correctly formatted graphs, but one of them is not a tree.*

Solution:

We will take a few separate steps in order to show that the language is in **L**.

1. Show that we can check that a graph is a tree in **L**.
2. Show that we can check if a given graph is of radius at most 2 in **L**.
3. Put it all together and show that the language is in **L**.

Assuming that the input is given the same way as in the assignment, we check that a graph is a tree by first checking that the number of edges is less than the number of vertices by one. Now we only have to check that the graph is connected. That can be done using an oracle for the problem of the path in a graph from vertices s to t . We apply it to every two vertices to make sure that the graph is indeed a tree. We know that the path can be checked in log space, therefore we can state if a graph is a tree in log space as well.

Next, stating that the graph is of radius at most two is easy, knowing it's a tree. We check for each vertex if the number of reachable vertices from the given vertex in two steps is n . This is log space as well. At last, after checking that both the graphs are trees and the first one has a center of radius at most 2, we will check the main condition. The algorithm is as follows:

For the sake of convenience, we will use a macro $\text{fn}(T)$ to denote putting the graph or other input arguments on the tape and running the $\text{fn}(\text{oracle})$. First, the list of oracles that we will be using:

$\text{IsTree}(T : \text{graph})$ - oracle for checking if a graph is a tree

$\text{IsRadiusAtMostTwo}(T : \text{graph})$ - oracle for checking if a tree is of radius at most two

$\text{IsCenterAtMostTwo}(T : \text{graph}, v : \text{vertex})$ - oracle for checking if the given vertex of a tree can reach all vertices in 2 steps.

$\text{NumNeighbours}(T : \text{graph}, v : \text{vertex})$ - oracle for counting the number of neighbours of a vertex in a graph, log space for obvious reasons

$\text{NumNeighboursWithDegree}(T : \text{graph}, v : \text{vertex}, d : \text{int})$ - oracle for counting the number of neighbours of a vertex with degree d in a graph, log space assuming that T is a tree (which it is in our case)

$\text{HighestDegreeNeighbour}(T : \text{graph}, v : \text{vertex})$ - oracle for finding the highest degree of a neighbour of a vertex in a graph, log space since T is a tree.

```

1 CheckL(T1: graph, T2: graph) -
2   if not IsTree(T1) or not IsTree(T2) then return false
3   if not IsRadiusAtMostTwo(T1) then return false
4   var x, y: vertex
5   var demand, supply, i, n1, n2 : int
6
7   for x in T1.vertices, y in T2.vertices do
8     if not IsCenterAtMostTwo(T1, x) then continue
9     if NumberNeighbours(T1, x) > NumberNeighbours(T2, y) then
10      continue
11    demand, supply, i = 0, 0, 0
12    while i < n - 1 do
13      n1 = NumberNeighboursWithDegree(T1, x, i)
14      n2 = NumberNeighboursWithDegree(T2, y, i)
15      if n2 >= n1 then
16        n2 -= n1
17      else do
18        demand += n1 - n2
19        n2 = 0
20      if demand < n2 then
21        n2 -= demand
22        demand = 0
23      else do
24        demand -= n2
25        n2 = 0
26      if i >= HighestDegreeNeighbour(T1, x) then
27        supply += n2
28      if supply < demand then continue
29      else do return true
30    return false

```

In the algorithm we brute force "centers" of both graphs and try to check if the given center of T2 has enough vertices of needed degrees to be a subgraph of T1 with a selected center. If it can, then the statement is true as we can remove all not needed vertices. The algorithm runs in log space as all the oracles are log space. Therefore, the language is in **L**.

Problem 2: Consider now a language consisting of pairs of graphs (T, G) where T is a tree of radius at most 2, G is a graph, and T is isomorphic to an induced subgraph of G . Prove that this language is **NP**-complete.

Solution:

The language is clearly in **NP**, since this is a special case of the induced subgraph isomorphism problem. We show hardness by a reduction from the k -clique problem. For every graph G and a clique size k , we will compute a log space instance (T', G') s.t. the instance is in the language iff the answer to the problem is positive.

Let G' be an inverse of G with an additional vertex connected to each other vertex. Let T' be a tree consisting of a vertex and its k neighbours. We will now argue that the pair

(T', G') is in the language iff the answer to the problem is positive.

$\Rightarrow$

Since the pair is in the language, it means that G' contains an empty induced subgraph on k vertices (by removing the center vertex), and since the added vertex can't be a node in the tree (it is connected to every other vertex), inverse of G contains an empty induced subgraph on k vertices, which is the same as saying that G contains a clique on k vertices as a subgraph.

$\Leftarrow$

k -clique in G implies k -empty in the inverse of G , which implies a k -star in G'